LILA BLACK JOURNEY VIEWER · HOW I BUILT THIS

From parquet files to the screen

How 1,243 raw game logs become the Overview, Journeys and Heatmaps pages. A Python pipeline does all the work once, before deploy. The browser only downloads small files and draws them.

1,243
Parquet files found
5 day folders, 8.4 MB

89,016
Rows read
88,653 kept after cleaning

796
Matches
3 maps, 6 dates

5.2 MB
Shipped to the browser
3.9 MB JSON + 1.3 MB minimaps

1 · RAW
Parquet + minimaps
1,243 files, 3 images, never in git

2 · BUILD
Python pipeline
Clean, project, aggregate. Run with `make data`

3 · OUTPUT
Static JSON + WebP
Committed to git, 5.2 MB

4 · HOST
Vercel
Static files only, no server code

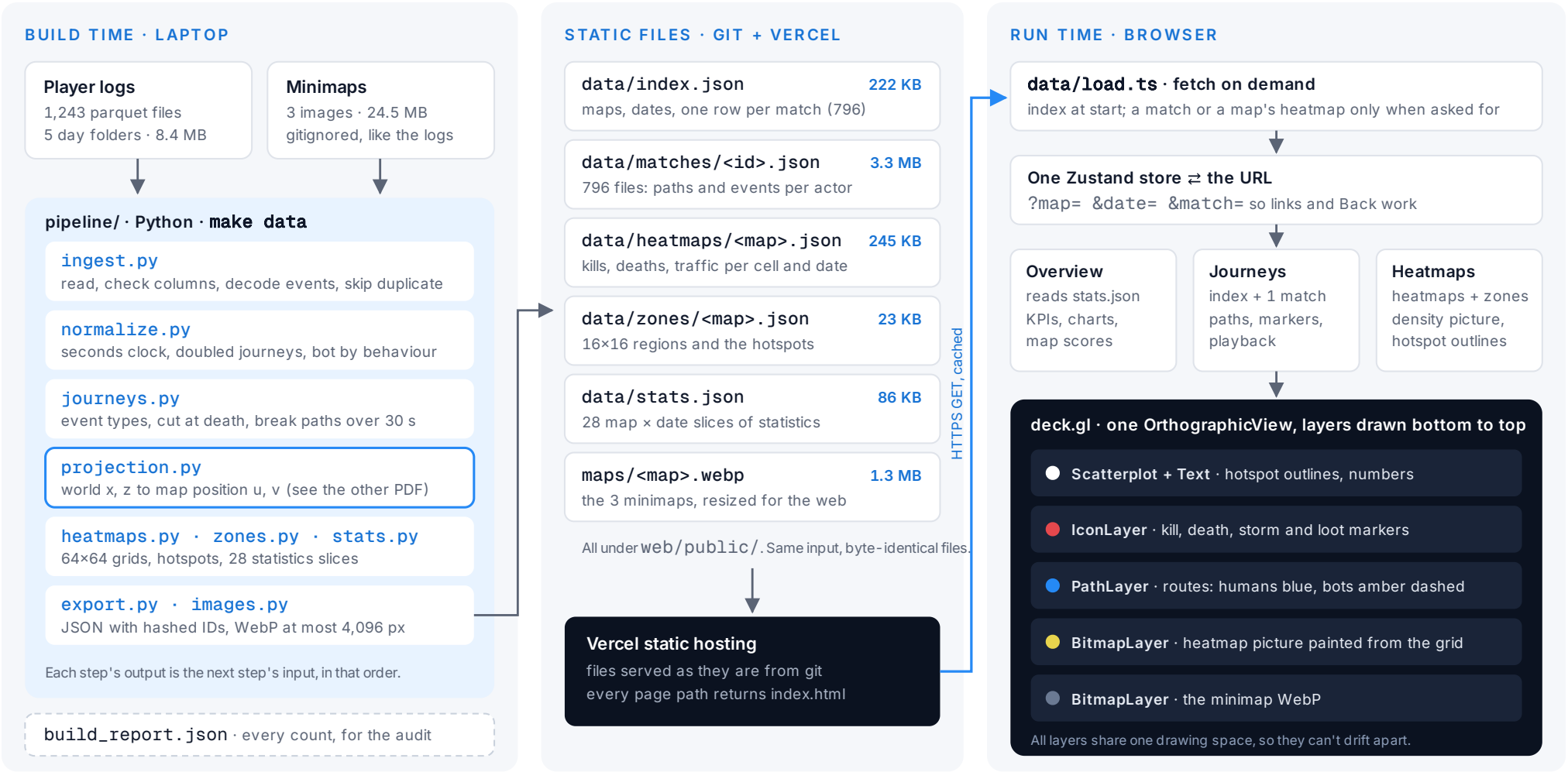
5 · DRAW
React + deck.gl
Loads files on demand and draws them

Every number in this document is computed by `pipeline/tools/explainer_pdfs.py` from the data and the generated files.

SYSTEM DESIGN

Three places the data lives, and what moves between them

Build time and run time are separate. Everything that needs the raw data happens on a laptop before deploy; the site only reads what the pipeline wrote.



INSIDE THE PIPELINE

Six steps, each fixing something the raw logs get wrong

The data README is wrong in places, so every rule below comes from checking the files. Nothing is dropped silently: every removed or flagged row is counted in `build_report.json`.

01

`ingest.py`

Reads every file, checks the 8 columns and their types, decodes the event name from bytes to text.

One file sits in two day folders: read once. 1,242 of 1,243 read, 0 failed.

02

`normalize.py`

Reads `ts` as Unix seconds and starts each match's clock at its first event. Decides bot or human.

3 journeys were logged twice in full: the copy goes. A file that logs `BotPosition` is a bot.

03

`journeys.py`

Gives each event a type (position, kill, death, storm death, loot) and cuts each journey at its first death.

173 positions were logged after the player died. Gaps over 30 s break the path (311 breaks).

04

`projection.py`

Turns world x and z into a map position from 0 to 1. The only place this happens.

The README's image size (1024 px) is wrong; positions don't depend on image size at all.

05

`heatmaps · zones · stats`

Counts kills and deaths per cell, spreads time along each step, finds hotspot regions, computes the statistics.

Only human journeys count. A hotspot needs events from at least 5 matches.

06

`export · images`

Writes the JSON files and the WebP minimaps, with no timestamps, so a rebuild gives identical bytes.

Player IDs are replaced by a short hash. Match files are swapped in only after all 796 are written.

Where the rows go

	Rows read from 1,242 files	89,016
−	Second copy of the 3 journeys logged twice in full	190
−	Positions logged after the player's first death (172 journeys)	173
=	**Rows kept and exported**	**88,653**

Kept, but flagged

- **17 journeys** have a bot-style numeric ID but log like the player client. Counted as human, flagged `id_mismatch`.
- **3 Kill/Killed pairs** in the same file and second. The death counts; the kill doesn't, and no cause is claimed.
- **2 late kills** logged after the player died. Kept with their timing marked uncertain.

What the 88,653 kept rows are



Positions 72,676 (82.0 %) · Loot 12,829 (14.5 %) · Kills 2,409 (2.7 %) · Deaths 700 (0.8 %) · Storm deaths 39

Who they belong to: 1,242 journeys



Human 798 (64.3 %) · Bot 444 (35.7 %)

A journey is one player (or bot) in one match: one parquet file. Most positions are the movement samples logged about every 5 s; they become the paths.

ONE EVENT, END TO END

Following a single kill from the parquet row to the pixel

A real row: a human on Lockdown kills a bot 69 s into match `00b34c64`. The player ID is shown as its hash, the only form that leaves the pipeline.

1 · RAW ROW

As stored in the file

ingest.py

```
user_id   ce4c905e…
match_id  00b34c64…
map_id    Lockdown
x -207.80
y 50.38
z 176.78
ts    1770760534
event b'BotKill'
```

2 · CLOCK

Seconds, not milliseconds

normalize.py

```
epoch_s = 1770760534
= 2026-02-10 21:55:34 UTC

t = epoch_s − first
    event in match
  = 69 s
```

3 · MEANING

Type, actor, flags

normalize.py · journeys.py

```
event     BotKill
category  kill
actor     human
          (no BotPosition)
pair      false
after_death false
```

4 · PROJECT

World to map position

projection.py

```
u = (x − (-500)) / 1,000
  = 0.2922
v = (z − (-500)) / 1,000
  = 0.6768

y (height) is ignored
```

5 · EXPORT

In the match file

export.py

```
matches/00b34c64.json
actors[].events[]:
{"t": 69, "type": "kill", "r
aw": "BotKill", "u": 0.2922,
"v": 0.6768, "after_death":
false, "kill_killed_pair": f
alse}
```

6 · DRAW

On the minimap

map/space.ts · IconLayer

```
drawing space
X = u × 1000
  = 292.2
Y = (1 − v) × 1000
  = 323.2

pixel on the 4,096 px
WebP: 1197, 1324
```

The timestamp trap

The column is typed as milliseconds. Read that way, this row would be at 1970-01-21 11:52:40 and the whole dataset would span a few minutes of 1970. As seconds, it falls on its folder's date.

Whose position is it?

An event's x and z are where the file's owner stood. So a kill marker sits where the killer stood, and a death marker where the victim fell.

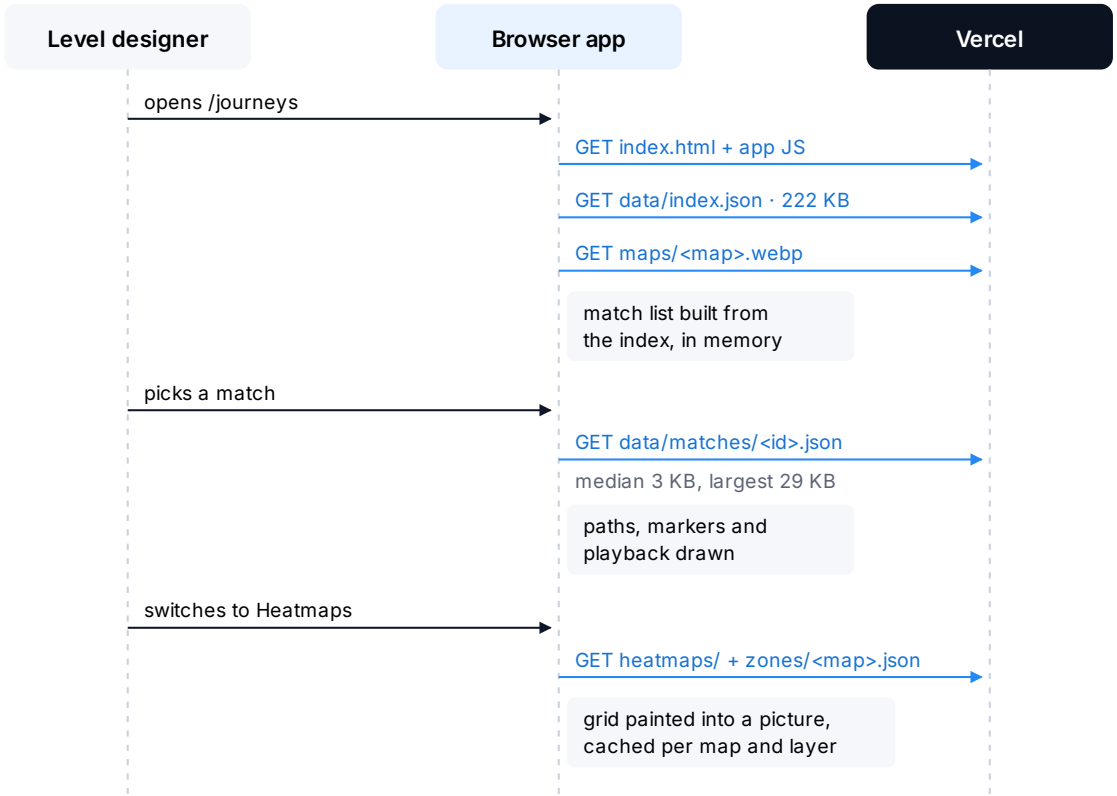
Why only u and v leave

The browser never sees world units, so paths, markers and heatmaps all come from the same converted numbers. The 4-decimal rounding is the only change.

RUN TIME

What the browser downloads, and when

Nothing is computed from raw data in the browser. It downloads the index once, then the one file a view needs. Filtering, sorting and drawing happen on files already in memory.



FILE	READ BY	SIZE
`index.json`	every map page, at start	222 KB
`matches/<id>.json`	Journeys, one per opened match	3 KB median
`heatmaps/<map>.json`	Heatmaps, one per map	245 KB for 3
`zones/<map>.json`	Heatmaps, hotspot outlines	23 KB for 3
`stats.json`	Overview and About	86 KB
`maps/<map>.webp`	every map view	1.3 MB for 3

Done in the browser

- Filtering by map and date, sorting the match list.
- Cutting each path at the playback time, every frame.
- Painting the heatmap grid into a soft picture (display only; no values change).
- Turning the cursor position back into world x, z for the readout.

Why there is no server

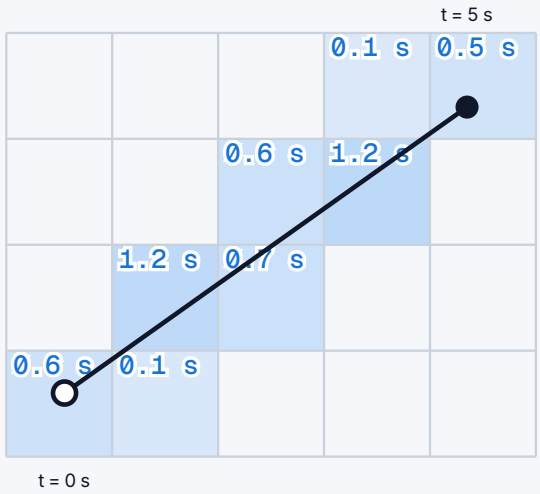
The data is fixed and every question the pages ask is known in advance, so the answers are files. No database to run, no API to keep alive, nothing that can fail on a reviewer's first visit.

AGGREGATION

How positions and events become a heatmap

Heatmaps are built from human journeys only. The 64×64 grid holds the numbers; the soft picture on screen is painted from it and never changes them.

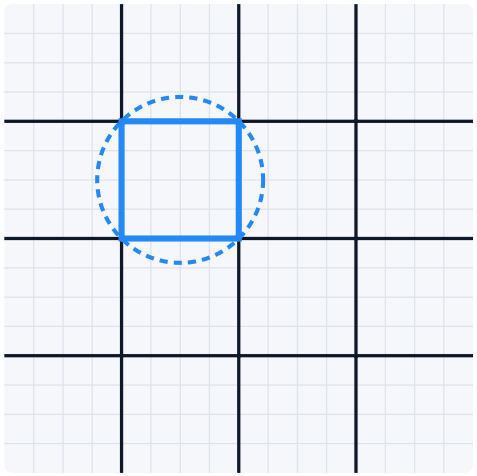
1 · Traffic: time along the route



Positions are about 5 s apart. Counting only the cell each sample lands in would leave the cells in between empty, so each step's time is spread along its straight line: here a 5 s step crosses 8 cells, split by the distance inside each (computed with the pipeline's own `trace_step`).

Steps over 30 s add nothing: where the player went is unknown.

2 · Kills and deaths: counts per cell

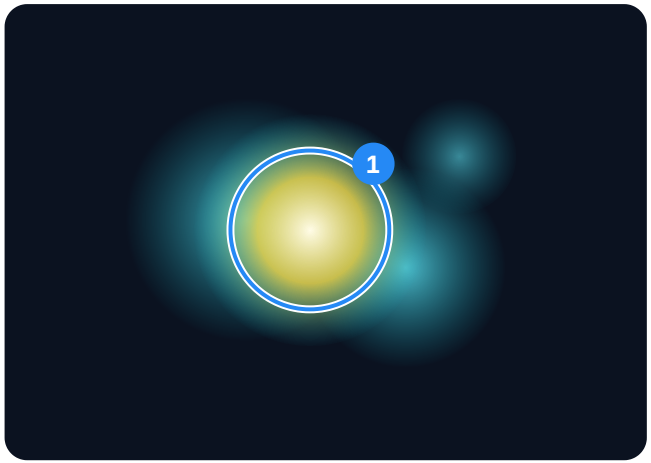


Each kill or death adds 1 to its cell. Thin lines: a corner of the 64×64 grid. Thick lines: 16×16 regions of 4×4 cells, which hotspots are judged on, because single cells mostly hold one event.

A region is a hotspot when

- events come from at least 5 different matches;
- no single match gives over 50 % of its value;
- it is among the 6 strongest that pass, per layer.

3 · In the browser: painted, not recomputed



- Each non-zero cell is painted as a soft bump (Gaussian, σ = 1 cell) whose colour comes from that cell's own value; neighbours are never added up, so no peak is invented.
- Traffic uses one teal to yellow palette on a log scale. Kills and deaths use four count classes: 1, 2–3, 4–9, 10+.
- Hotspot outlines come straight from `zones/<map>.json`; hovering reads the grid, not the pixels.

GUARANTEES AND LIMITS

What the design makes sure of, and what it can't

The pipeline is the single source of every number on screen. These are the properties that follow from that, and the gaps the data itself leaves.

Reproducible

Same input, same bytes

No build timestamps and a fixed sort order, so `make data` on the same raw files gives byte-identical output. A change in a diff is a change in the data or the code.

Safe rebuilds

Nothing half-written

Match files are written to a staging folder, checked against the index (one file per match, right schema), and only then swapped in. The old folder goes last.

One source

Pages can't disagree

One clock, one bot rule, one coordinate function, and statistics computed once into `stats.json`. The pages read; they don't recalculate.

Private

No real IDs leave

Player IDs are hashed before export, raw logs are gitignored, and `make check-ids` scans everything trackable for real IDs.

Audited

Every exclusion counted

`build_report.json` lists files skipped, rows removed by reason, flags, journeys by type and the path breaks, so each cleaning step can be checked.

Fast first visit

Small, on demand

5.2 MB in total, but a visit loads the index and one map first. A match file is 3 KB at the median.

Limit

Recorded players only

Most matches have one recorded human. Everything describes what was logged, not every player in every match.

Limit

Straight lines between samples

Paths and traffic join samples about 5 s apart with straight lines. The real route between them is unknown.

Limit

No extraction event

A journey with no recorded death probably extracted, but the logs never say so; anything built on that is labelled inferred.